

Многослойный персептрон

Выполнил: Шабардин Егор, группа 09-321

Содержание

1	Постановка задачи, данные и их анализ	2
1.1	Постановка задачи	2
1.2	Данные и их анализ	2
1.3	Разделение и предобработка	3
2	Устройство многослойного персептрона	5
2.1	Прямое распространение	5
2.2	Функции потерь	6
2.3	Обратное распространение и градиентный спуск	7
2.4	Инициализация весов	8
3	Описание библиотеки и разбор примера	9
3.1	Устройство библиотеки	9
3.2	Пример обучения сети	9

1 Постановка задачи, данные и их анализ

1.1 Постановка задачи

В проекте решается задача медицинской диагностики. По данным, нужно определить, является опухоль злокачественной (класс М) или доброкачественной (класс В). Поскольку классов два, это задача бинарной классификации. Для ее решения нужно реализовать многослойный персептрон — простейшую полносвязную нейронную сеть. Все алгоритмы написаны самостоятельно на Python с помощью библиотеки линейной алгебры NumPy. Библиотека pandas используется только для чтения CSV-файлов, matplotlib — для построения графиков. Требования к работе:

- провести анализ данных и разделить их на обучающую и валидационную выборки;
- реализовать сеть, в которой по умолчанию не меньше двух скрытых слоев, а на выходном слое стоит функция softmax;
- обучить сеть с помощью алгоритма обратного распространения ошибки и градиентного спуска;
- выводить значения функции потерь на каждой эпохе и построить кривые обучения.

Каждый пациент описывается вектором признаков $x \in \mathbb{R}^{30}$. Сеть получает этот вектор на вход и выдает два числа (p_B, p_M) — вероятности того, что опухоль доброкачественная и злокачественная. Эти числа неотрицательны и в сумме дают единицу. Ответом сети считается класс с большей вероятностью. Правильные ответы для обучения записываются в виде векторов из нулей и единицы (one-hot кодирование): класс В — это вектор $(1, 0)$, класс М — вектор $(0, 1)$. Обучение сети заключается в подборе ее весов так, чтобы выходы сети были как можно ближе к правильным ответам. Насколько они близки, измеряется функцией потерь, которую мы минимизируем. Качество обученной модели оценивается по значению функции потерь и по точности (ассигасу) — доле правильно классифицированных объектов.

1.2 Данные и их анализ

Данные находятся в файле `data.csv`. В нем 569 строк и 32 столбца, строки заголовков нет. Первый столбец — номер пациента, второй — диагноз (М или В), остальные 30 — числовые признаки, полученные по результатам анализов. Данные проверены на пропуски — их нет, поэтому заполнять или удалять ничего не нужно. Затем проанализировано, сколько объектов каждого класса: 357 доброкачественных и 212 злокачественных. Классы немного несбалансированы, но не сильно, так что специальных мер не требуется. Признаки сильно различаются по масштабу (табл. 1).

Таблица 1. Статистики некоторых признаков

Признак	Среднее	Ст. откл.	Мин.	Макс.
feature_1	14,13	3,52	6,98	28,11
feature_4	654,89	351,91	143,50	2501,00
feature_10	0,063	0,007	0,050	0,097
feature_15	0,007	0,003	0,002	0,031
feature_24	880,58	569,36	185,20	4254,00

Перед обучением признаки нужно привести к одному масштабу. Также построена матрица корреляций между признаками (рис. 1). На ней видно, что некоторые признаки почти повторяют друг друга. Всего нашлась 21 пара признаков с корреляцией больше 0.9 по модулю. Скорее всего, такие признаки описывают одну и ту же величину разными способами.

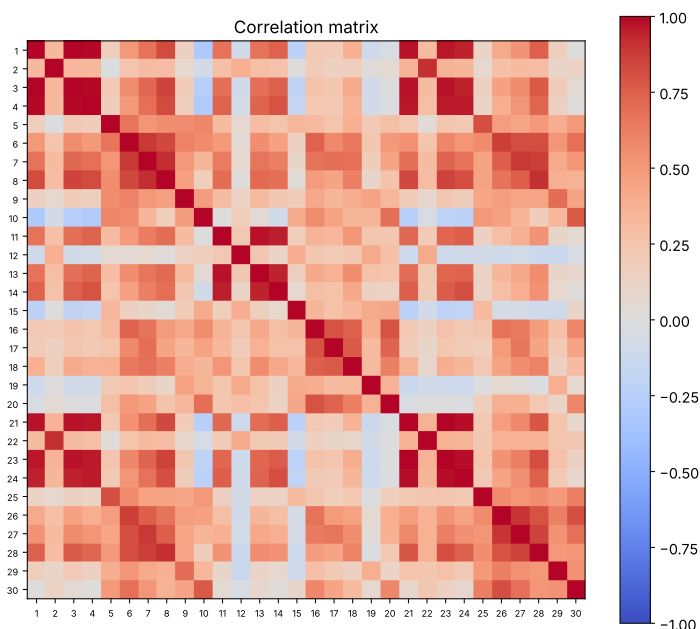


Рис. 1. Матрица корреляций признаков.

1.3 Разделение и предобработка

Данные разделены на две части в отношении 70 к 30: обучающая выборка (398 объектов) и валидационная (171 объект). Для этого строки перемешиваются случайной перестановкой, первые 70% идут в обучающую выборку, остальные — в валидационную. Результат сохраняется в файлы `train.csv` и `valid.csv`. Чтобы разбиение было одинаковым при каждом запуске, генератор случайных чисел создается с фиксированным зерном 42. Это же зерно используется при обучении сети для начальных весов и перемешивания данных, поэтому весь эксперимент можно точно повторить. Для приведения признаков к одному масштабу используется стандартизация: из каждого признака

вычитается его среднее и результат делится на стандартное отклонение:

$$\tilde{x}_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}.$$

После этого у каждого признака среднее равно 0, а стандартное отклонение — 1. Важно, что μ_j и σ_j считаются только по обучающей выборке, а потом с этими же значениями преобразуется и валидационная. Если считать их по всем данным, то в обучение попадет информация из валидационной выборки, и оценка качества получится завышенной. Диагноз переводится в число, а затем в one-hot вектор длины 2.

2 Устройство многослойного персептрона

2.1 Прямое распространение

Многослойный персептрон состоит из нескольких полносвязных слоев, идущих друг за другом. Полносвязный означает, что каждый нейрон слоя связан со всеми нейронами предыдущего слоя. Слой с n_{in} входами и n_{out} выходами хранит матрицу весов W размера $n_{\text{in}} \times n_{\text{out}}$ и вектор смещений b длины n_{out} . Слой сначала вычисляет линейную комбинацию входов, а затем применяет к ней функцию активации φ .

Удобно подавать в сеть сразу несколько объектов. Если записать B объектов строками матрицы X , то вычисления в слое l записываются так:

$$Z^{(l)} = A^{(l-1)}W^{(l)} + b^{(l)}, \quad A^{(l)} = \varphi^{(l)}(Z^{(l)}), \quad A^{(0)} = X, \quad (1)$$

где вектор $b^{(l)}$ прибавляется к каждой строке. Выход последнего слоя $A^{(L)}$ — это ответ сети. Такой проход по слоям от входа к выходу называется прямым распространением. Функция активации нужна, чтобы сеть была нелинейной. Если ее убрать, то несколько слоев подряд — это просто произведение матриц, то есть одно линейное преобразование, и сеть не будет сильнее обычной линейной модели. В библиотеке реализованы следующие функции активации:

Функция	$\varphi(z)$	$\varphi'(z)$
Identity	z	1
ReLU	$\max(0, z)$	1 при $z > 0$, иначе 0
Leaky ReLU	$\max(\alpha z, z)$	1 при $z > 0$, иначе α
Sigmoid	$\sigma(z) = \frac{1}{1 + e^{-z}}$	$\sigma(z)(1 - \sigma(z))$
Tanh	$\tanh z$	$1 - \tanh^2 z$

Производные сигмоиды и тангенса выражаются через значение самой функции, поэтому при прямом проходе сохраняется выход активации, и потом его можно сразу использовать при вычислении производной. В скрытых слоях используется ReLU. У сигмоиды производная не больше 0.25, поэтому при прохождении через много слоев градиент быстро уменьшается. У ReLU производная равна 1 для положительных входов, и такой проблемы нет.

На выходном слое нужна функция, которая превращает выходы сети в вероятности. Для этого используется функция активации softmax:

$$s_k(z) = \frac{e^{z_k}}{\sum_j e^{z_j}}.$$

Все s_k положительны и в сумме дают 1. Если входы большие, то e^{z_k} может переполниться. Поэтому перед вычислением из всех z_k вычитается максимальное значение. Результат от этого не меняется. В отличие от остальных функций, каждый выход softmax зависит от всех входов, поэтому ее производная — это матрица. Обозначим $S = \sum_j e^{z_j}$. Если $k = j$, то

$$\frac{\partial s_k}{\partial z_k} = \frac{e^{z_k} S - e^{z_k} e^{z_k}}{S^2} = s_k(1 - s_k),$$

а если $k \neq j$, то

$$\frac{\partial s_k}{\partial z_j} = -\frac{e^{z_k} e^{z_j}}{S^2} = -s_k s_j.$$

Пусть $g_k = \partial \mathcal{L} / \partial s_k$ — градиент функции потерь по выходам softmax. Тогда по правилу дифференцирования сложной функции получаем, что

$$\frac{\partial \mathcal{L}}{\partial z_j} = \sum_k g_k \frac{\partial s_k}{\partial z_j} = s_j \left(g_j - \sum_k g_k s_k \right). \quad (2)$$

2.2 Функции потерь

Для обучения используется категориальная кросс-энтропия. Если y — правильный ответ в виде one-hot вектора, а p — выход сети, то для батча из B объектов

$$\mathcal{L}_{\text{CE}} = -\frac{1}{B} \sum_{i=1}^B \sum_k y_{ik} \ln p_{ik}.$$

Так как в векторе y только одна единица, для каждого объекта остается одно слагаемое — $\ln p$ правильного класса. Чем больше вероятность, которую сеть дала правильному классу, тем меньше потери. Производная по выходам сети:

$$\frac{\partial \mathcal{L}_{\text{CE}}}{\partial p_{ik}} = -\frac{1}{B} \frac{y_{ik}}{p_{ik}}.$$

Чтобы не получить $\ln 0$, значения p ограничиваются снизу числом 10^{-8} . Если подставить эту производную в формулу (2) и учесть, что $\sum_k y_{ik} = 1$, получается результат:

$$\frac{\partial \mathcal{L}}{\partial z_{ij}} = \frac{1}{B} p_{ij} \left(-\frac{y_{ij}}{p_{ij}} + \sum_k y_{ik} \right) = \frac{1}{B} (p_{ij} - y_{ij}).$$

То есть градиент на выходном слое — это разность между предсказанием и правильным ответом. В реализации softmax и функция потерь — отдельные классы, но при последовательном вызове их методов backward получается именно эта формула.

По заданию модель нужно оценить бинарной кросс-энтропией:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^N [y_i \ln p_i + (1 - y_i) \ln(1 - p_i)],$$

где y_i — метка (0 или 1), а p_i — вероятность класса М. В библиотеке эта функция применяется ко всем элементам матрицы выходов и усредняется.

Кроме этого, в библиотеке есть среднеквадратичная ошибка (MSE) и средняя абсолютная ошибка (MAE):

$$\mathcal{L}_{\text{MSE}} = \frac{1}{n} \sum (p - y)^2, \quad \mathcal{L}_{\text{MAE}} = \frac{1}{n} \sum |p - y|.$$

Они чаще используются в задачах регрессии, поэтому в этой работе не применялись.

2.3 Обратное распространение и градиентный спуск

Чтобы обучить сеть, нужно знать производные функции потерь по всем весам. Их вычисляет алгоритм обратного распространения ошибки. Идея в том, чтобы идти по слоям от выхода к входу и на каждом шаге применять правило дифференцирования сложной функции. Пусть в слой l пришел градиент $G = \partial \mathcal{L} / \partial A^{(l)}$ по его выходу. Для последнего слоя его считает функция потерь. Далее выполняются три шага.

1. Градиент проходит через функцию активации. Для поэлементных функций это просто умножение на производную:

$$\Delta = \frac{\partial \mathcal{L}}{\partial Z^{(l)}} = G \odot \varphi'(Z^{(l)}), \quad (3)$$

где $\odot$ — поэлементное умножение. Для softmax используется формула (2).

2. Считаются градиенты по параметрам слоя. Так как $Z = AW + b$, то

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = (A^{(l-1)})^\top \Delta, \quad \frac{\partial \mathcal{L}}{\partial b^{(l)}} = \sum_{i=1}^B \Delta_i. \quad (4)$$

Здесь вход слоя $A^{(l-1)}$ берется из прямого прохода, поэтому слой его запоминает.

3. Считается градиент, который передаётся в предыдущий слой:

$$\frac{\partial \mathcal{L}}{\partial A^{(l-1)}} = \Delta (W^{(l)})^\top. \quad (5)$$

Эти три шага повторяются для всех слоев от последнего к первому. После того как градиенты посчитаны, веса обновляются методом градиентного спуска:

$$W \leftarrow W - \eta \frac{\partial \mathcal{L}}{\partial W}, \quad b \leftarrow b - \eta \frac{\partial \mathcal{L}}{\partial b}, \quad (6)$$

где η — шаг обучения (learning rate). Градиент показывает направление, в котором функция потерь растет быстрее всего, поэтому шаг в обратную сторону ее уменьшает. Если η слишком большой, обучение может расходиться, а если слишком маленький — идет очень медленно. Используется стохастический градиентный спуск по мини-батчам.

В начале каждой эпохи обучающая выборка перемешивается и делится на батчи по B объектов, и после каждого батча веса обновляются. Так обновление происходит много раз за эпоху, и обучение идет быстрее, чем если считать градиент по всей выборке сразу.

2.4 Инициализация весов

Перед обучением веса нужно чем-то заполнить. Нулями этого делать нельзя, так как все нейроны слоя будут считать одно и то же, получают одинаковые градиенты и так и останутся одинаковыми. Поэтому веса задаются случайными числами, а смещения — нулями. Важно также выбрать разброс случайных весов. Если веса слишком большие или слишком маленькие, сигнал при прохождении через слои будет расти или затухать. В библиотеке реализованы две известные схемы:

- инициализация Ксавье, которая подходит для sigmoid и tanh: дисперсия весов $\frac{2}{n_{\text{in}} + n_{\text{out}}}$;
- инициализация Кайминга, которая подходит для ReLU: дисперсия весов $\frac{2}{n_{\text{in}}}$.

Каждая схема есть в двух вариантах — с нормальным распределением и с равномерным распределением на отрезке $[-a, a]$. В работе для скрытых слоев с ReLU используется инициализация Кайминга, а для выходного слоя с softmax — инициализация Ксавье.

3 Описание библиотеки и разбор примера

3.1 Устройство библиотеки

Библиотека `multilayer_perceptron` состоит из семи модулей:

Модуль	Содержание
<code>registry.py</code>	реестр компонентов по имени
<code>activation_functions.py</code>	Identity, ReLU, LeakyReLU, Sigmoid, Tanh, Softmax
<code>initializers.py</code>	Xavier и Kaiming (нормальные и равномерные)
<code>loss_functions.py</code>	MSE, MAE, бинарная и категориальная кросс-энтропия
<code>layers.py</code>	полносвязный слой <code>DenseLayer</code>
<code>optimizers.py</code>	стохастический градиентный спуск
<code>network.py</code>	класс <code>Network</code> : предсказание, оценка, обучение

Для функций активации, функций потерь, инициализаторов и оптимизаторов сделаны абстрактные базовые классы с общим интерфейсом. Каждый конкретный класс при объявлении автоматически записывается в реестр под своим именем. Поэтому объект можно создать просто по строке. Сеть настраивается строками, а чтобы добавить новую функцию активации, достаточно написать один новый класс. Функции активации и потерь при прямом проходе запоминают то, что понадобится для производных, и реализуют формулы из главы 2. Слой `DenseLayer` делает прямой проход по формуле (1) и обратный по формулам (3)–(5). Оптимизатор обновляет веса всех слоев по формуле (6). Класс `Network` собирает все вместе: слои, функцию потерь и оптимизатор. Его метод `fit` обучает сеть по эпохам, после каждой эпохи выводит потери на обучающей и валидационной выборках.

3.2 Пример обучения сети

Разберем, как работает обучение в скрипте `train.py`. Сначала он загружает обучающую и валидационную выборки, берет из них 30 признаков, переводит диагноз в one-hot вектор и стандартизует признаки, как описано в главе 1. Получаются матрицы размера (398, 30) и (171, 30). Затем из трех полносвязных слоев собирается сеть:

Слой	Размер	Активация	Инициализация
1 (скрытый)	$30 \rightarrow 16$	ReLU	Kaiming normal
2 (скрытый)	$16 \rightarrow 8$	ReLU	Kaiming normal
3 (выходной)	$8 \rightarrow 2$	Softmax	Xavier normal

В сети два скрытых слоя и всего $30 \cdot 16 + 16 + 16 \cdot 8 + 8 + 8 \cdot 2 + 2 = 650$ параметров. Для обучения используется категориальная кросс-энтропия и стохастический градиентный спуск с шагом $\eta = 0.01$. Размер батча — 32, число эпох — 100, зерно генератора — 42.

При таком размере батча одна эпоха — это 13 обновлений весов. Во время обучения на каждой эпохе выводятся значения функции потерь. Некоторые из них, вместе с точностью, приведены в табл. 2. После обучения строятся кривые обучения (рис. 2).

Таблица 2. Значения метрик на отдельных эпохах

Эпоха	loss	val_loss	acc	val_acc
1	1,0299	1,0413	0,342	0,327
5	0,5118	0,5532	0,844	0,784
10	0,3108	0,3605	0,920	0,883
20	0,1712	0,2103	0,947	0,947
50	0,0909	0,1259	0,975	0,959
100	0,0603	0,1233	0,985	0,959

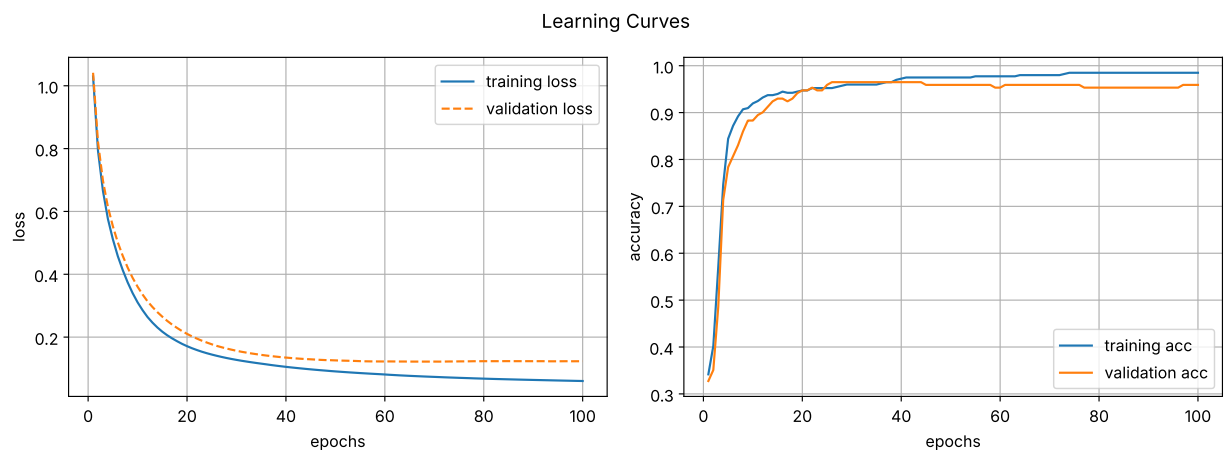


Рис. 2. Кривые обучения: функция потерь (слева) и точность (справа)

Примерно после 50-й эпохи потери на валидационной выборке перестают уменьшаться и остаются около 0.123, а на обучающей продолжают медленно падать до 0.060. Это значит, что сеть начинает немного подстраиваться под обучающие данные. Но переобучение слабое: потери на валидации не растут, и точность на ней не падает.